

FORWARD DEPLOYED AI ENGINEERING

Capstone Project

Dataset Guide

What is in each file, what every field means, and how each set may be used

Read this before you load anything. The three ticket sets serve different purposes and using the wrong one for the wrong thing will invalidate your evaluation.

REFERENCE · WEEK ONE

The four files you have been given

File	Contents	What it is for
development_tickets.json	500 labelled tickets with full history	Understanding the problem, developing your classifier and router, building everything.
validation_tickets.json	80 labelled tickets, same schema	Checking your own performance as you build. Use as often as you like.
ground_truth_responses.json	200 reference answers written by senior agents	A standard for what a good answer looks like, and a comparison set for response quality.
documentation.json	29 knowledge base articles	The corpus your retrieval layer searches. This is CloudServe's support documentation.

There is a fourth ticket set you do not have

Your work is finally assessed against a hidden evaluation set of 120 tickets that is not distributed with this pack and that you will not see before submission. It is drawn from the same population and uses the same schema, so a system that performs well on your validation set should perform comparably on it. This is why tuning against a set until the number looks good is self-defeating: it is not the set that decides your grade.

How this changes what you should do

- Develop against the 500 development tickets. They are yours to use however you like.
- Check yourself against the 80 validation tickets as often as you want. They are not the final measure, so there is no penalty for repeated use.
- Report your validation figures in your report, and state plainly that they come from validation rather than from the hidden set.
- Build your evaluation harness so that it can be pointed at any file with this schema. It will be run against the hidden set exactly as you wrote it.

Your harness must accept an input path

After submission, your harness is run against the hidden set. If it only works against a file path you hardcoded, it cannot be run, and that is treated as a failure of acceptance criterion A9. Take an input path and an output path as arguments.

1 Ticket schema

Development and validation tickets share this structure. The hidden set uses the same shape, so anything you build against these will work against it.

```
{
  "ticket_id": "DEV-0001",
  "channel": "email",
  "subject": "Cannot log in to the console",
  "body": "I have been trying to sign in ...",
  "received_at": "2026-03-14T09:22:00Z",
  "customer_id": "CUST-1042",
  "customer_name": "Priya Sharma",
  "customer_tier": "business",
  "customer_region": "europe",
  "language_fluency": "fluent",

  "labels": {
    "intent": "authentication_failure",
    "urgency": "high",
    "expected_route": "auto_respond",
    "answerable_from_docs": true,
    "expected_doc_ids": ["DOC-AUTH-001"],
    "must_not_auto_respond": false
  },

  "history": {
    "first_contact_resolution": true,
    "resolution_time_minutes": 12,
    "csat_rating": 4,
    "escalated": false,
    "repeat_contact": false
  }
}
```

What each label is for

Field	Use it to measure
labels.intent	Classification precision and recall, per class.
labels.urgency	Whether your urgency prediction is usable for prioritisation.
labels.expected_route	Routing accuracy, and the cost of your threshold choice in both directions.
labels.answerable_from_docs	Whether your system correctly recognises questions it cannot ground.
labels.expected_doc_ids	Citation accuracy: did you cite the document that actually answers this?
labels.must_not_auto_respond	A safety check. Auto-responding to any of these is a governance failure, not a scoring loss.
history.*	The current baseline. This is what CloudServe achieves today, and what you are trying to improve on.

On the history block

These fields record what actually happened when a human handled the ticket. They are the baseline you compare against, not labels for your system to reproduce. A system that faithfully predicts a historical CSAT of 3.2 has learned to imitate an operation that is failing, which is not the objective.

2 Documentation schema

```
{
  "doc_id": "DOC-AUTH-001",
  "title": "Resolving invalid credential errors on login",
  "category": "authentication",
  "applies_to": "Console, CLI, SDK",
  "content": "# Resolving invalid credential errors ...",    markdown
  "related_docs": ["DOC-AUTH-002", "DOC-AUTH-004"],
  "last_reviewed_days_ago": 0
}
```

Each article follows the same internal structure: a title, an applies-to line, a symptoms list, common causes, numbered resolution steps and a notes section. That regularity is worth considering when you decide how to chunk the corpus, because splitting inside a resolution sequence tends to produce passages that retrieve well but read as incomplete.

Coverage

The corpus covers eight categories: authentication, deployment, api, performance, billing, data, security, account, integration and onboarding. Not every intent has a corresponding article, and that is deliberate. Feature requests and unclear requests have no documentation to retrieve, and a system that manufactures an answer for them is doing the wrong thing.

3 Ground truth response schema

```
{
  "ticket_id": "DEV-0042",
  "intent": "rate_limit",
  "expected_doc_ids": ["DOC-API-001"],
  "reference_response": "Thanks for reporting this. Rate limits ...",
  "must_mention": ["per organisation", "backoff"],    points a correct answer covers
  "must_not_claim": ["a refund has been issued", ...],  claims that would be wrong or unsafe
  "written_by": "senior_support_agent"
}
```

The must_mention and must_not_claim arrays give you something concrete to check automatically. An answer that omits everything in the first list is probably incomplete; an answer that makes any claim in the second is a governance problem rather than a quality one.

4 Composition of the development set

Some figures to save you counting, though you should verify them yourself as part of Stage 1 rather than taking them on trust.

Dimension	Composition
Size	500 tickets
Intent classes	22, unevenly distributed, as real queues are
Channels	Email is the largest share, then chat, then documentation comments, then forum
Customer tiers	Standard is roughly half, business roughly a third, enterprise the remainder
Language fluency	Around a quarter are marked non-fluent
Answerable from documentation	Around three quarters
Expected routing	Roughly two thirds auto-respond, one third escalate

Two things in this table are worth pausing on

The proportion answerable from existing documentation, and the size of the non-fluent segment. Both connect directly to things said in the stakeholder interviews, and both have consequences for what you should build and for what your fairness audit will find.

The fairness segments

Your governance framework asks you to compare quality across customer groups. The fields that make that possible are `customer_tier`, `customer_region` and `language_fluency`. Segment on each, compare resolution quality and citation accuracy, and report what you find honestly. A difference you discovered and reported is worth considerably more than a table showing no variation anywhere, which usually means the analysis was not sensitive enough to detect it.

5 Loading the data

```
import json
from pathlib import Path

DATA = Path('data')

dev = json.loads((DATA / 'development_tickets.json').read_text())
val = json.loads((DATA / 'validation_tickets.json').read_text())
docs = json.loads((DATA / 'documentation.json').read_text())
truth = json.loads((DATA / 'ground_truth_responses.json').read_text())

# index the reference answers by ticket for quick comparison
```

```
truth_by_id = {t['ticket_id']: t for t in truth}

# your harness must accept paths rather than hardcoding them,
# because it will be run against a file you have not seen
def run_evaluation(input_path: str, output_path: str) -> None:
    tickets = json.loads(Path(input_path).read_text())
    ...
```